

100 câu hỏi vấn đáp vào vị trí DEV tại Viettel năm 2025



Chuyên đề 1: Sắp xếp, Tìm kiếm, và Độ quy

Câu 1: Câu hỏi: Độ phức tạp thời gian (Time Complexity) là gì? Hãy phân biệt độ phức tạp $O(N)$, $O(N^2)$, và $O(\log N)$ với một ví dụ cho mỗi loại.

Lời đáp (Gợi ý): Độ phức tạp thời gian mô tả cách thời gian chạy của thuật toán tăng lên theo kích thước đầu vào (N).

- $O(N)$ (Tuyến tính): Tăng trưởng tỷ lệ thuận với N . Ví dụ: Duyệt qua một danh sách.
- $O(N^2)$ (Bậc hai): Tăng trưởng theo bình phương của N . Ví dụ: Thuật toán Sắp xếp nổi bọt (**Bubble Sort**).
- $O(\log N)$ (Logarit): Tăng trưởng rất chậm. Ví dụ: Thuật toán **Tìm kiếm nhị phân (Binary Search)**.

Câu 2: Câu hỏi: Giải thích sự khác biệt giữa **Quick Sort** và **Merge Sort**. Thuật toán nào được ưa chuộng hơn trong các hệ thống lớn và vì sao?

Lời đáp (Gợi ý):

- **Quick Sort:** Sắp xếp tại chỗ (in-place) và sử dụng phân hoạch (**Partition**). Tốt nhất $O(N\log N)$, tệ nhất $O(N^2)$.
- **Merge Sort:** Sắp xếp không tại chỗ (cần bộ nhớ phụ $O(N)$) và sử dụng **Chia để trị (Divide and Conquer)**. Luôn có độ phức tạp $O(N\log N)$. **Merge Sort** thường được ưa chuộng hơn trong các hệ thống lớn hoặc đa luồng vì nó ổn định (**stable**) và đảm bảo độ phức tạp thời gian **luôn là $O(N\log N)$** (xử lý song song tốt hơn).

Câu 3: Câu hỏi: Mô tả cách hoạt động của **Tìm kiếm nhị phân (Binary Search)**. Điều kiện tiên quyết để sử dụng thuật toán này là gì?

Lời đáp (Gợi ý): **Binary Search** hoạt động bằng cách liên tục chia đôi không gian tìm kiếm. Nó so sánh giá trị cần tìm với phần tử ở giữa mảng. Nếu không khớp, loại bỏ nửa mảng không chứa giá trị đó. **Điều kiện tiên quyết:** Dữ liệu phải được **sắp xếp (Sorted)** trước khi tìm kiếm.

Câu 4: Câu hỏi: Đệ quy (Recursion) là gì? Khi nào nên ưu tiên dùng đệ quy hơn vòng lặp? Nêu rủi ro của đệ quy.

Lời đáp (Gợi ý): **Đệ quy** là kỹ thuật gọi lại chính hàm đó để giải quyết một bài toán con. Ưu tiên dùng khi cấu trúc dữ liệu mang tính đệ quy (ví dụ: **Duyệt cây, Đồ thị**) hoặc bài toán có định nghĩa đệ quy rõ ràng (ví dụ: **Fibonacci, Giai thừa**). **Rủi ro:** Gây ra lỗi **Stack Overflow** (tràn bộ nhớ ngăn xếp) nếu không có **Điều kiện dừng (Base Case)** hoặc điều kiện dừng không bao giờ đạt được.

Câu 5: Câu hỏi: **Memoization** là gì và nó liên quan như thế nào đến Đệ quy? Cho một ví dụ.

Lời đáp (Gợi ý): **Memoization** là kỹ thuật tối ưu hóa bằng cách **lưu trữ kết quả** của các lời gọi hàm đệ quy đã được tính toán trước đó và trả về kết quả đã lưu khi cùng đầu vào xuất hiện lại. Nó được dùng để tránh tính toán lặp lại trong các bài toán **Đệ quy có sự chồng chéo của bài toán con (Overlapping Subproblems)**, ví dụ điển hình là tính toán số Fibonacci lớn.

Chuyên đề 2: Danh sách liên kết, Ngăn xếp, Hàng đợi (Linked Lists, Stack, Queue)

Câu 6: Câu hỏi: Danh sách liên kết đơn (Singly Linked List) khác gì so với Mảng (Array) về mặt cấu trúc và hiệu năng?

Lời đáp (Gợi ý):

- **Cấu trúc:** Mảng là các phần tử lưu trữ liên tiếp trong bộ nhớ. Linked List là các **Node** chứa dữ liệu và **con trỏ** trỏ đến Node tiếp theo.
- **Hiệu năng:**
 - Truy cập ngẫu nhiên (Random Access): Mảng $O(1)$, Linked List $O(N)$.
 - Chèn/Xóa ở vị trí bất kỳ: Mảng $O(N)$, Linked List $O(1)$ (nếu đã có con trỏ đến Node trước đó).

Câu 7: Câu hỏi: Giải thích về **Ngăn xếp (Stack)** và **Hàng đợi (Queue)** theo nguyên tắc hoạt động. Nêu ứng dụng thực tế của mỗi loại.

Lời đáp (Gợi ý):

- **Stack (Ngăn xếp):** Hoạt động theo nguyên tắc **LIFO (Last-In, First-Out)**—Vào sau ra trước. Ứng dụng: Quản lý lời gọi hàm (**Call Stack**), thao tác **Undo/Redo** trong phần mềm.
- **Queue (Hàng đợi):** Hoạt động theo nguyên tắc **FIFO (First-In, First-Out)**—Vào trước ra trước. Ứng dụng: Quản lý tác vụ chờ (**Task Scheduling**), bộ đệm (Buffering), **Breadth-First Search (BFS)**.

Câu 8: Câu hỏi: Mục đích của **Danh sách liên kết đôi (Doubly Linked List)** là gì so với Danh sách liên kết đơn?

Lời đáp (Gợi ý): Danh sách liên kết đôi có thêm con trỏ '**previous**' (trở về Node trước đó). Mục đích là cho phép **duyệt ngược (Backward Traversal)** và thực hiện thao tác **Xóa Node** trong thời gian $O(1)$ mà không cần con trỏ đến Node trước đó (vì ta có thể dùng con trỏ 'previous').

Câu 9: Câu hỏi: Làm thế nào để sử dụng Stack để kiểm tra tính hợp lệ của dấu ngoặc đơn (Parentheses) trong một biểu thức?

Lời đáp (Gợi ý): Duyệt qua biểu thức:

1. Nếu gặp dấu ngoặc **mở** ((, {, [), đẩy nó vào Stack.
2. Nếu gặp dấu ngoặc **đóng** (), },]), lấy phần tử trên cùng ra khỏi Stack và kiểm tra xem nó có khớp với dấu ngoặc đóng hiện tại không. Nếu không khớp hoặc Stack rỗng, biểu thức không hợp lệ.
3. Cuối cùng, nếu Stack **rỗng**, biểu thức là hợp lệ.

Câu 10: Câu hỏi: Giả sử bạn cần tạo bộ đệm (cache) cho API. Bạn sẽ dùng cấu trúc dữ liệu nào để thực hiện cơ chế **Least Recently Used (LRU) Cache**?

Lời đáp (Gợi ý): Để triển khai **LRU Cache** hiệu quả, cần kết hợp hai cấu trúc:

1. **Hash Map (Bảng băm):** Để lưu trữ Key-Value (Key là ID cache, Value là Node) cho phép truy cập $O(1)$.
2. **Doubly Linked List (Danh sách liên kết đôi):** Để theo dõi thứ tự sử dụng. Node được truy cập gần đây nhất được di chuyển lên đầu danh sách.

Chuyên đề 3: Cây và Đồ thị (Trees and Graphs)

Câu 11: Câu hỏi: Nêu sự khác biệt chính giữa **Cây (Tree)** và **Đồ thị (Graph)**.

Lời đáp (Gợi ý):

- **Đồ thị:** Là tập hợp các Đỉnh (Vertices) và Cạnh (Edges). Cạnh có thể tạo thành **chu trình (cycles)**, và không có ràng buộc về số lượng cạnh.
- **Cây:** Là một loại Đồ thị **vô hướng (undirected)** và **không có chu trình (acyclic)**. Nó có một gốc (Root) duy nhất và có tính phân cấp. Cây N đỉnh luôn có N-1 cạnh.

Câu 12: Câu hỏi: Trình bày về ba kiểu duyệt cây nhị phân (Binary Tree Traversal) cơ bản: Inorder, Preorder, và Postorder.

Lời đáp (Gợi ý):

1. **Inorder (Trung thứ tự):** Trái -> Gốc -> Phải. (Duyệt **BST** sẽ cho ra kết quả sắp xếp).
2. **Preorder (Tiền thứ tự):** Gốc -> Trái -> Phải. (Dùng để **Copy** cây hoặc tạo **Prefix Expression**).
3. **Postorder (Hậu thứ tự):** Trái -> Phải -> Gốc. (Dùng để **Xóa** cây hoặc tạo **Postfix Expression**).

Câu 13: Câu hỏi: Heap (**Đống**) là gì? Phân biệt Max Heap và Min Heap. Ứng dụng chính của cấu trúc này là gì?

Lời đáp (Gợi ý): Heap là một cây nhị phân gần như hoàn chỉnh (**complete binary tree**) tuân theo tính chất Heap Order Property.

- **Max Heap:** Giá trị của Gốc luôn lớn hơn hoặc bằng giá trị của các nút con.
- **Min Heap:** Giá trị của Gốc luôn nhỏ hơn hoặc bằng giá trị của các nút con. **Ứng dụng:** Triển khai **Priority Queue (Hàng đợi ưu tiên)**, thuật toán **Heap Sort**.

Câu 14: Câu hỏi: Giải thích hai thuật toán duyệt đồ thị cơ bản: **BFS (Breadth-First Search)** và **DFS (Depth-First Search)**. Trường hợp nào nên dùng BFS?

Lời đáp (Gợi ý):

- **BFS (Tìm kiếm theo chiều rộng):** Sử dụng **Queue**, duyệt tất cả các Node ở cùng một mức trước khi chuyển sang mức sâu hơn (Layer by Layer).
- **DFS (Tìm kiếm theo chiều sâu):** Sử dụng **Stack** (hoặc Đệ quy), đi sâu vào một nhánh hết mức có thể trước khi quay lại. **Nên dùng BFS** khi cần tìm **đường đi ngắn nhất** (Shortest Path) giữa hai Node trong đồ thị **không có trọng số**.

Câu 15: Câu hỏi: Cây tìm kiếm nhị phân cân bằng (Balanced Binary Search Tree - Ví dụ: AVL, Red-Black) có ưu điểm gì so với BST thông thường?

Lời đáp (Gợi ý): BST thông thường có thể bị thoái hóa thành **Danh sách liên kết (Skewed Tree)** nếu dữ liệu được chèn theo thứ tự tăng hoặc giảm, dẫn đến độ phức tạp của các thao tác (Tìm kiếm, Chèn, Xóa) là $O(N)$. **BST cân bằng** đảm bảo rằng độ cao của cây luôn được giữ ở mức $O(\log N)$ bằng các thao tác **xoay (rotations)**, duy trì hiệu năng $O(\log N)$ cho tất cả các thao tác.

Chuyên đề 4: Thuật toán Tối ưu và Thiết kế (Dynamic, Greedy, Backtracking)

Câu 16: Câu hỏi: Dynamic Programming (Quy hoạch động) là gì? Hai điều kiện tiên quyết để áp dụng DP là gì?

Lời đáp (Gợi ý): Quy hoạch động là phương pháp giải quyết các bài toán phức tạp bằng cách chia chúng thành các bài toán con đơn giản hơn và giải từng bài toán con đó, sau đó kết hợp các lời giải lại. Hai điều kiện tiên quyết:

1. **Tối ưu cấu trúc con (Optimal Substructure):** Lời giải tối ưu của bài toán lớn được xây dựng từ lời giải tối ưu của các bài toán con.
2. **Các bài toán con chồng chéo (Overlapping Subproblems):** Cùng một bài toán con được giải lặp đi lặp lại.

Câu 17: Câu hỏi: Phân biệt Dynamic Programming (DP) và Greedy Algorithm (Tham lam).

Lời đáp (Gợi ý):

- **Greedy:** Tại mỗi bước, thuật toán đưa ra **lựa chọn tốt nhất cục bộ** (locally optimal choice) với hy vọng đạt được lời giải tối ưu toàn cục. **Không cần kiểm tra lại** các bước trước. (Ví dụ: Đổi tiền xu).
- **DP:** Phải xem xét lời giải của **tất cả** các bài toán con chồng chéo và **đảm bảo tối ưu** ở mọi bước. Nó có tính toàn cục (globally optimal). (Ví dụ: Bài toán Cái túi - Knapsack).

Câu 18: Câu hỏi: Backtracking (Quay lui) là gì? Nêu một ứng dụng của Backtracking.

Lời đáp (Gợi ý): Backtracking là một thuật toán tìm kiếm tất cả các lời giải bằng cách xây dựng một lời giải từng bước. Nếu một lựa chọn tại bước hiện tại dẫn đến bế tắc (không thể hoàn thành lời giải), thuật toán sẽ **quay lại** (backtrack) bước trước đó và thử một lựa chọn khác. **Ứng dụng:** Giải các bài toán ràng buộc (Constraint Satisfaction Problems) như **Giải Sudoku**, **Bài toán N-Hậu (N-Queens)**, Tìm tất cả đường đi.

Câu 19: Câu hỏi: Trong DP, phân biệt phương pháp Top-Down (Memoization) và Bottom-Up (Tabulation).

Lời đáp (Gợi ý):

- **Top-Down (Memoization):** Bắt đầu từ bài toán lớn và gọi đệ quy các bài toán con (tự nhiên). Kết quả của bài toán con được lưu trong một mảng (memo).
- **Bottom-Up (Tabulation):** Bắt đầu từ các bài toán con cơ sở (base cases) nhỏ nhất, sau đó sử dụng các kết quả này để tính toán lên lời giải của các bài toán con lớn hơn, lưu trữ kết quả trong một bảng (table) mà không cần đệ quy.

Câu 20: Câu hỏi: Bài toán Two Pointers (Hai con trỏ) là gì? Nó giúp tối ưu độ phức tạp như thế nào?

Lời đáp (Gợi ý): Two Pointers là kỹ thuật sử dụng hai con trỏ (thường là chỉ mục mảng) để duyệt qua một cấu trúc dữ liệu. Hai con trỏ có thể di chuyển đồng thời hoặc di chuyển ngược chiều nhau. Nó giúp tối ưu hóa thuật toán từ $O(N^2)$ (ví dụ: dùng hai vòng lặp lồng nhau) xuống $O(N)$ (tuyến tính) bằng cách tận dụng cấu trúc đã sắp xếp (ví dụ: tìm cặp số có tổng bằng K).

Chuyên đề 5: Cấu trúc dữ liệu nâng cao và Vấn đề thực tiễn

Câu 21: Câu hỏi: Hash Table (Bảng băm) là gì? Làm thế nào nó xử lý các va chạm (Collisions)?

Lời đáp (Gợi ý): Hash Table là cấu trúc dữ liệu ánh xạ **Key** thành **Value** bằng cách sử dụng **hàm băm (hash function)** để tính toán vị trí lưu trữ (chỉ mục). Xử lý va chạm:

1. **Separate Chaining (Chuỗi riêng biệt):** Lưu trữ các phần tử cùng chỉ mục băm vào một **Linked List** tại vị trí đó.
2. **Open Addressing (Địa chỉ mở):** Tìm kiếm vị trí trống tiếp theo trong bảng (ví dụ: Linear Probing, Quadratic Probing).

Câu 22: Câu hỏi: Giải thích về Trie (Prefix Tree). Nó được sử dụng để giải quyết vấn đề gì?

Lời đáp (Gợi ý): Trie là một cấu trúc dữ liệu cây đặc biệt dùng để lưu trữ và truy xuất chuỗi (string) nơi các Node chia sẻ các tiền tố (prefix) chung. **Ứng dụng: Tìm kiếm tiền tố (Prefix Search), tự động hoàn thành (Autocomplete), và kiểm tra chính tả** nhanh hơn nhiều so với Hash Table.

Câu 23: Câu hỏi: Disjoint Set Union (Union-Find) là gì? Nó được dùng để giải quyết loại bài toán nào?

Lời đáp (Gợi ý): Disjoint Set Union (còn gọi là Union-Find) là cấu trúc dữ liệu quản lý các tập hợp (set) không giao nhau. Nó hỗ trợ hai thao tác chính:

1. **Find:** Xác định tập hợp mà một phần tử thuộc về (tìm ra đại diện/gốc).
2. **Union:** Hợp nhất hai tập hợp thành một. **Ứng dụng:** Kiểm tra xem hai Node có thuộc cùng một thành phần liên thông (Connected Component) trong đồ thị không, và các thuật toán tìm **cây bao trùm nhỏ nhất (Minimum Spanning Tree)** như Kruskal.

Câu 24: Câu hỏi: Giả sử bạn cần tìm kiếm một chuỗi con (substring) trong một chuỗi văn bản rất dài. Thuật toán nào nên được ưu tiên và vì sao?

Lời đáp (Gợi ý): Nên ưu tiên thuật toán **Knuth-Morris-Pratt (KMP)**. **Lý do:** KMP có độ phức tạp thời gian $O(M+N)$ (với M là chiều dài mẫu, N là chiều dài văn bản), đây là độ phức tạp **tuyến tính** và nhanh hơn nhiều so với thuật toán brute-force ($O(MN)$) vì nó tránh kiểm tra lại các ký tự đã biết.

Câu 25: Câu hỏi: Topological Sort là gì? Điều kiện tiên quyết để áp dụng thuật toán này là gì?

Lời đáp (Gợi ý): Topological Sort là một sắp xếp tuyến tính (linear ordering) của các đỉnh trong một đồ thị có hướng, sao cho với mọi cạnh có hướng $U \rightarrow V$, đỉnh U luôn xuất hiện trước đỉnh V trong thứ tự sắp xếp. **Điều kiện tiên quyết:** Đồ thị phải là **Đồ thị có hướng không có chu trình (Directed Acyclic Graph - DAG)**.

Chuyên đề 6: Tư duy và Tối ưu (Critical Thinking and Optimization)

Câu 26: Câu hỏi: Khi được yêu cầu tối ưu hóa một thuật toán, anh/chị sẽ bắt đầu từ đâu?

Lời đáp (Gợi ý):

1. **Đo lường (Profiling):** Xác định **điểm nghẽn (bottleneck)** chính xác (phần nào của code chiếm nhiều thời gian nhất) bằng công cụ profiling. Đừng tối ưu hóa mù quáng.
2. **Phân tích độ phức tạp:** Đảm bảo thuật toán đang được sử dụng có độ phức tạp tốt nhất có thể (ví dụ: từ $O(N^2)$ xuống $O(N \log N)$ hoặc $O(N)$).
3. **Cấu trúc dữ liệu:** Xem xét liệu có thể chuyển đổi sang Cấu trúc Dữ liệu hiệu quả hơn không (ví dụ: dùng Hash Map thay cho Array để tìm kiếm).

Câu 27: Câu hỏi: Làm thế nào để giải quyết vấn đề **Integer Overflow (Tràn số nguyên)** trong lập trình cạnh tranh hoặc hệ thống thực tế?

Lời đáp (Gợi ý):

1. **Sử dụng kiểu dữ liệu lớn hơn:** Chuyển sang sử dụng `long long` (C/C++) hoặc `long` (Java/C#) để có phạm vi lưu trữ lớn hơn.
2. **Kiểm tra điều kiện:** Trước khi thực hiện phép toán (ví dụ: phép nhân), kiểm tra xem kết quả có vượt quá giới hạn của kiểu dữ liệu hiện tại không.
3. **Modulus (Phép chia lấy dư):** Trong một số bài toán (đặc biệt là DP), sử dụng phép toán modulo để giữ kết quả trong phạm vi cho phép.

Câu 28: Câu hỏi: Giải thích về kỹ thuật **Sliding Window (Cửa sổ trượt)**. Mục đích chính của nó là gì?

Lời đáp (Gợi ý): Sliding Window là kỹ thuật được dùng trên Mảng hoặc Chuỗi để chuyển đổi độ phức tạp từ $O(N^2)$ xuống $O(N)$. Nó duy trì một "cửa sổ" (một phạm vi con) và trượt cửa sổ đó qua cấu trúc dữ liệu, thêm một phần tử ở cuối và loại bỏ một phần tử ở đầu. **Mục đích chính:** Tìm kiếm các tập hợp con/chuỗi con có kích thước cố định hoặc động thỏa mãn điều kiện tối ưu (ví dụ: tìm tổng con lớn nhất, chuỗi con dài nhất không lặp lại).

Câu 29: Câu hỏi: Trong các bài toán tối ưu hóa, làm thế nào để anh/chị chứng minh rằng một thuật toán Tham lam (Greedy) là đúng?

Lời đáp (Gợi ý): Chứng minh thuật toán Tham lam là đúng thường yêu cầu hai bước:

1. **Cấu trúc con tối ưu (Optimal Substructure):** Chứng minh rằng lời giải tối ưu của bài toán lớn chứa lời giải tối ưu của các bài toán con.

2. **Lựa chọn tham lam (Greedy Choice Property):** Chứng minh rằng lựa chọn tham lam tại bước hiện tại **luôn** là một phần của lời giải tối ưu toàn cục (ví dụ: dùng phương pháp chứng minh bằng phản chứng).

Câu 30: Câu hỏi: Nếu thời gian chạy của thuật toán của anh/chị không thể chấp nhận được, nhưng đã là $O(N \log N)$, giải pháp tiếp theo là gì?

Lời đáp (Gợi ý):

1. **Tối ưu hằng số:** Tối ưu hóa các hệ số ẩn trong $O(N \log N)$ (ví dụ: giảm số lần gọi hàm, tối ưu hóa code bên trong vòng lặp).
2. **Xử lý song song (Parallel Processing):** Nếu có thể, chia công việc thành các tác vụ độc lập và xử lý song song trên nhiều lõi/luồng (Ví dụ: sử dụng **MapReduce** hoặc các thư viện đa luồng).
3. **Tận dụng Caching/Pre-computation:** Tính toán trước các kết quả và lưu vào cache hoặc Hash Table.
4. **Sử dụng Ngôn ngữ cấp thấp hơn:** Nếu cần thiết, viết lại các phần quan trọng (bottlenecks) bằng ngôn ngữ cấp thấp hơn (ví dụ: C/C++) để tăng tốc độ.

Chuyên đề 7: Lập trình Hướng đối tượng (OOP Core)

Câu 31: Câu hỏi: Giải thích bốn trụ cột cơ bản của Lập trình Hướng đối tượng (OOP) và tại sao **Tính đóng gói (Encapsulation)** lại quan trọng.

Lời đáp (Gợi ý): Bốn trụ cột: **Tính Đóng gói (Encapsulation)**, **Tính Kế thừa (Inheritance)**, **Tính Đa hình (Polymorphism)**, và **Tính Trừu tượng (Abstraction)**. **Tính đóng gói** quan trọng vì nó **che giấu dữ liệu** (data hiding) và logic bên trong đối tượng, chỉ phơi bày các phương thức công khai (public methods) để tương tác. Điều này giúp bảo vệ tính toàn vẹn của dữ liệu và cho phép thay đổi triển khai bên trong mà không ảnh hưởng đến code bên ngoài.

Câu 32: Câu hỏi: Phân biệt giữa **Đa hình tại thời điểm biên dịch (Compile-time Polymorphism)** và **Đa hình tại thời điểm chạy (Runtime Polymorphism)**, kèm ví dụ.

Lời đáp (Gợi ý):

- **Compile-time Polymorphism (Đa hình tĩnh):** Xảy ra qua cơ chế **Nạp chồng phương thức (Method Overloading)**, nơi nhiều phương thức cùng tên nhưng khác nhau về tham số/kiểu dữ liệu. Trình biên dịch biết chính xác phương thức nào sẽ được gọi.
- **Runtime Polymorphism (Đa hình động):** Xảy ra qua cơ chế **Ghi đè phương thức (Method Overriding)**, sử dụng **Kế thừa** và **Tham chiếu lớp cha** trỏ đến **Đối tượng lớp con**. Việc gọi phương thức được quyết định khi chương trình đang chạy.

Câu 33: Câu hỏi: Giải thích vai trò của **Interface** và **Abstract Class**. Khi nào nên sử dụng Interface thay vì Abstract Class?

Lời đáp (Gợi ý):

- **Abstract Class:** Dùng để định nghĩa một "**là một loại**" (**is-a relationship**) chung cho các lớp con. Có thể chứa cả phương thức trừu tượng (abstract methods) và phương thức có thân (concrete methods), và trường dữ liệu.
- **Interface:** Dùng để định nghĩa một "**có thể làm**" (**can-do contract**). Chỉ chứa các khai báo phương thức (từ Java 8/C# 8 có thể có phương thức mặc định). Nên dùng **Interface** khi muốn một lớp có thể thực hiện **nhiều hợp đồng** (đa kế thừa hành vi, vì nhiều ngôn ngữ không hỗ trợ đa kế thừa lớp) hoặc khi muốn tách biệt định nghĩa hành vi khỏi chi tiết triển khai.

Câu 34: Câu hỏi: Delegation (Ủy quyền) là một mô hình thiết kế thay thế cho Kế thừa. Anh/chị hãy giải thích mô hình này và ưu điểm của nó.

Lời đáp (Gợi ý): Delegation là mô hình trong đó một đối tượng (Delegator) truyền trách nhiệm thực hiện một tác vụ cho một đối tượng khác (Delegate). Về mặt cấu trúc, nó sử dụng **Thành phần/Hợp thành (Composition)** thay vì Kế thừa. **Ưu điểm:** Giúp tránh được các vấn đề của Kế thừa (ví dụ: chặt chẽ hóa cao, Kế thừa lớp cơ sở lớn), thúc đẩy nguyên tắc **Composition over Inheritance** (Ưu tiên Hợp thành hơn Kế thừa), làm cho code linh hoạt hơn và dễ bảo trì hơn.

Câu 35: Câu hỏi: Immutable Class là gì? Nêu các bước cơ bản để tạo một Immutable Class (ví dụ: trong Java/C#).

Lời đáp (Gợi ý): Immutable Class là một lớp mà các đối tượng của nó không thể thay đổi trạng thái sau khi được tạo. Các bước cơ bản:

1. Đánh dấu lớp là **final** (hoặc tương đương) để ngăn chặn Kế thừa.
2. Đánh dấu tất cả các trường dữ liệu là **final** và **private**.
3. Chỉ cung cấp **Constructor** (không cung cấp setter).
4. Nếu lớp chứa các đối tượng có thể thay đổi (Mutable objects), phải tạo **bản sao sâu (deep copy)** trong constructor và trong getter.

Chuyên đề 8: Quản lý Bộ nhớ (Memory Management)

Câu 36: Câu hỏi: Phân biệt giữa Stack Memory và Heap Memory trong quản lý bộ nhớ ứng dụng.

Lời đáp (Gợi ý):

- **Stack Memory:** Dùng cho **biến cục bộ (local variables)** và **lời gọi hàm (function calls)**. Bộ nhớ được cấp phát và thu hồi tự động theo thứ tự **LIFO**. Nhanh nhưng kích thước nhỏ.
- **Heap Memory:** Dùng cho **đối tượng (objects)** và dữ liệu được cấp phát động. Bộ nhớ phải được giải phóng thủ công (C/C++) hoặc bởi **Bộ thu gom rác (Garbage Collector)** (Java/Python). Chậm hơn nhưng kích thước lớn và linh hoạt.

Câu 37: Câu hỏi (C++): Giải thích sự khác biệt giữa **con trỏ (Pointer)** và **tham chiếu (Reference)** trong C++.

Lời đáp (Gợi ý):

- **Pointer:** Là một biến lưu trữ **địa chỉ bộ nhớ** của một biến khác. Có thể trỏ tới `nullptr` và có thể bị thay đổi để trỏ tới đối tượng khác. Cần giải phóng thủ công.
- **Reference:** Là một **biệt danh (alias)** cho một biến đã tồn tại. Phải được khởi tạo ngay lập tức và không thể trỏ tới đối tượng khác sau khi khởi tạo. Không cần giải phóng.

Câu 38: Câu hỏi (Java/.NET): Vai trò của **Garbage Collector (GC)** là gì? GC trong Java/C# hoạt động dựa trên nguyên tắc nào?

Lời đáp (Gợi ý): GC là cơ chế tự động giải phóng bộ nhớ trên Heap không còn được chương trình sử dụng nữa. GC hoạt động dựa trên nguyên tắc **Reachability (Khả năng truy cập)**. Nó xác định các đối tượng không còn được truy cập từ bất kỳ **Gốc (Root)** nào (ví dụ: Stack, biến tĩnh) và đánh dấu chúng để thu hồi.

Câu 39: Câu hỏi (C++): Smart Pointers (Con trỏ thông minh) là gì? Nêu ưu điểm của `std::unique_ptr` và `std::shared_ptr`.

Lời đáp (Gợi ý): **Smart Pointers** là các đối tượng wrapper cho con trỏ thô, tự động quản lý vòng đời của bộ nhớ.

- **`std::unique_ptr`:** Đảm bảo **quyền sở hữu độc quyền** đối với tài nguyên. Bộ nhớ được giải phóng khi `unique_ptr` ra khỏi phạm vi (scope). Ngăn ngừa rò rỉ bộ nhớ đơn giản.
- **`std::shared_ptr`:** Sử dụng bộ đếm tham chiếu (**Reference Count**) để theo dõi số lượng con trỏ cùng trỏ tới tài nguyên đó. Bộ nhớ chỉ được giải phóng khi bộ đếm bằng 0.

Câu 40: Câu hỏi (Python): Python quản lý bộ nhớ như thế nào? Giải thích về cơ chế **Reference Counting (Đếm tham chiếu)**.

Lời đáp (Gợi ý): Python sử dụng cơ chế **Reference Counting** kết hợp với **Garbage Collector** bổ sung để xử lý chu trình tham chiếu. **Reference Counting** là mỗi đối tượng có một bộ đếm theo dõi số lượng tham chiếu trỏ đến nó. Khi bộ đếm giảm về 0, đối tượng ngay lập tức được giải phóng.

Chuyên đề 9: Xử lý Ngoại lệ và Luồng (Exception and Threading)

Câu 41: Câu hỏi: Phân biệt giữa **Lỗi (Error)** và **Ngoại lệ (Exception)**. Tại sao việc xử lý ngoại lệ lại cần thiết?

Lời đáp (Gợi ý):

- **Error:** Là các vấn đề nghiêm trọng, thường nằm ngoài tầm kiểm soát của ứng dụng (ví dụ: `StackOverflowError`, `OutOfMemoryError`).
- **Exception:** Là các tình huống không mong muốn nhưng có thể dự đoán và xử lý được (ví dụ: `NullPointerException`, `FileNotFoundException`). **Xử lý ngoại lệ** cần thiết để:

Ngăn ứng dụng bị sập, cung cấp thông báo lỗi thân thiện cho người dùng, và cho phép chương trình phục hồi hoặc thoát ra một cách an toàn.

Câu 42: Câu hỏi (Java/.NET): Phân biệt **Checked Exception** và **Unchecked Exception** (Runtime Exception).

Lời đáp (Gợi ý):

- **Checked Exception:** Buộc phải được xử lý (bằng `try-catch`) hoặc khai báo (`throws/throw`) trong chữ ký phương thức tại **thời điểm biên dịch** (compiler). Dành cho các vấn đề mà ứng dụng có thể khôi phục.
- **Unchecked Exception (Runtime Exception):** Không bắt buộc phải xử lý tại thời điểm biên dịch. Thường là lỗi logic lập trình (ví dụ: `NullPointerException`, `ArrayIndexOutOfBoundsException`).

Câu 43: Câu hỏi: Deadlock (Tắc nghẽn) là gì trong môi trường đa luồng? Nêu bốn điều kiện cần thiết để xảy ra Deadlock.

Lời đáp (Gợi ý): **Deadlock** là tình huống hai hay nhiều luồng bị mắc kẹt vĩnh viễn, mỗi luồng đang chờ tài nguyên (resource) đang bị luồng khác nắm giữ. Bốn điều kiện cần thiết (Coffman Conditions):

1. **Mutual Exclusion (Độc quyền):** Ít nhất một tài nguyên không thể được chia sẻ.
2. **Hold and Wait (Giữ và Chờ):** Một luồng giữ tài nguyên và chờ tài nguyên khác.
3. **No Preemption (Không chiếm quyền):** Tài nguyên không thể bị lấy đi trừ khi luồng nắm giữ tự nguyện giải phóng.
4. **Circular Wait (Chờ vòng tròn):** Một chuỗi luồng chờ nhau theo vòng tròn.

Câu 44: Câu hỏi: Giải thích về khái niệm **Thread Safety (An toàn luồng)** và cách đảm bảo nó.

Lời đáp (Gợi ý): **Thread Safety** là khả năng một chương trình hoặc đoạn code hoạt động đúng khi được thực thi đồng thời bởi nhiều luồng. Đảm bảo Thread Safety bằng cách:

1. **Đồng bộ hóa (Synchronization):** Sử dụng `lock`, `synchronized block/method` để bảo vệ các đoạn code truy cập vào tài nguyên chia sẻ (Critical Section).
2. **Sử dụng đối tượng bất biến (Immutability):** Dùng các lớp/đối tượng không thể thay đổi trạng thái.
3. **Sử dụng các cấu trúc dữ liệu an toàn luồng** (ví dụ: `ConcurrentHashMap` trong Java).

Câu 45: Câu hỏi: Semaphore và Mutex là gì? Chúng khác nhau cơ bản như thế nào trong việc kiểm soát truy cập tài nguyên?

Lời đáp (Gợi ý): Cả hai đều là cơ chế đồng bộ hóa để kiểm soát truy cập vào tài nguyên chia sẻ.

- **Mutex (Mutual Exclusion):** Là một khóa nhị phân (binary lock). Chỉ có thể có **một luồng** được sở hữu khóa tại một thời điểm. Dùng để bảo vệ một phần code khỏi bị truy cập đồng thời.

- **Semaphore:** Là một cơ chế đếm (counting mechanism). Nó cho phép **một số lượng** luồng nhất định (count > 1) truy cập vào tài nguyên giới hạn đồng thời.

Chuyên đề 10: Chi tiết Ngôn ngữ (Cú pháp và Thư viện)

Câu 46: Câu hỏi (Python): Python là ngôn ngữ **Dynamic Typed (gõ động)**. Điều này khác gì so với C++ hay Java (**Static Typed**)? Ưu nhược điểm là gì?

Lời đáp (Gợi ý):

- **Dynamic Typed:** Kiểu dữ liệu của một biến được kiểm tra và xác định tại **thời điểm chạy (runtime)**. (Ví dụ: `x = 10`, sau đó `x = "hello"`).
- **Static Typed:** Kiểu dữ liệu của một biến được xác định tại **thời điểm biên dịch (compile-time)**. Ưu điểm (Dynamic): **Nhanh phát triển**, code ngắn gọn hơn. Nhược điểm: **Dễ gây ra lỗi runtime** do lỗi kiểu dữ liệu không bị phát hiện sớm.

Câu 47: Câu hỏi (Java/C#): Từ khóa **static** được sử dụng để làm gì? Nêu sự khác biệt giữa `static variable` và `instance variable`.

Lời đáp (Gợi ý): Từ khóa **static** được dùng để định nghĩa thành viên (biến, phương thức, khối) **thuộc về chính lớp đó**, chứ không phải thuộc về một đối tượng cụ thể.

- **Instance Variable:** Mỗi đối tượng (instance) có một bản sao riêng của biến này.
- **Static Variable:** Chỉ có **một bản sao duy nhất** của biến được chia sẻ bởi tất cả các đối tượng của lớp đó.

Câu 48: Câu hỏi (C++): Từ khóa **const** trong C++ có những ứng dụng nào?

Lời đáp (Gợi ý): `const` được dùng để chỉ định rằng một giá trị không thể bị thay đổi.

1. **const variable:** Biến có giá trị không đổi.
2. **const pointer:** Con trỏ không thể thay đổi địa chỉ mà nó trỏ tới.
3. **const data:** Dữ liệu mà con trỏ trỏ tới không thể thay đổi.
4. **const method:** Phương thức không được phép thay đổi trạng thái (trường dữ liệu) của đối tượng.

Câu 49: Câu hỏi: Trình bày về **Generic Programming (Lập trình chung)** và lợi ích của nó.

Lời đáp (Gợi ý): **Generic Programming** cho phép viết các thuật toán và cấu trúc dữ liệu hoạt động với **mọi kiểu dữ liệu** mà không làm mất an toàn kiểu dữ liệu (Type Safety). Lợi ích:

1. **Tái sử dụng code:** Viết code một lần cho nhiều kiểu dữ liệu (ví dụ: một lớp `List<T>` hoạt động với `List<Integer>` hoặc `List<String>`).
2. **An toàn kiểu dữ liệu:** Tránh được lỗi kiểu dữ liệu tại thời điểm chạy.

Câu 50: Câu hỏi (Java): **Final** (trong Java) và **Const** (trong C++) khác nhau như thế nào?

Lời đáp (Gợi ý):

- **final (Java):** Có thể áp dụng cho **lớp** (ngăn kế thừa), **phương thức** (ngăn ghi đè), và **biến** (biến được gán giá trị duy nhất). Chủ yếu là về **nguyên tắc thiết kế** và **quản lý vòng đời**.
- **const (C++):** Chủ yếu là về **tính bất biến** của giá trị và sự an toàn bộ nhớ. Áp dụng cho biến và hàm để chỉ định rằng chúng sẽ không thay đổi dữ liệu mà chúng tham chiếu.

Chuyên đề 11: Mạng và Giao thức (Networking and Protocols)

Câu 51: Câu hỏi: Giải thích sự khác biệt chính giữa **TCP (Transmission Control Protocol)** và **UDP (User Datagram Protocol)**. Ứng dụng phổ biến của mỗi giao thức là gì?

Lời đáp (Gợi ý):

- **TCP:** Là giao thức **hướng kết nối (connection-oriented)**. Đảm bảo dữ liệu được truyền đi **đáng tin cậy (reliable)**, đúng thứ tự, và có cơ chế kiểm soát lỗi/luồng. Ứng dụng: Duyệt web (HTTP/HTTPS), FTP, Email (SMTP).
- **UDP:** Là giao thức **phi kết nối (connectionless)**. Không đảm bảo tính tin cậy hay thứ tự. Nhanh hơn nhưng không có kiểm soát lỗi. Ứng dụng: Truyền phát video/âm thanh trực tuyến (Streaming), DNS, Game trực tuyến.

Câu 52: Câu hỏi: **REST API** là gì? Nêu bốn động từ HTTP (HTTP Verbs) cơ bản được sử dụng trong thiết kế REST.

Lời đáp (Gợi ý): **REST (Representational State Transfer) API** là một kiến trúc thiết kế dịch vụ web, sử dụng giao thức HTTP và tập trung vào các tài nguyên (resources). Bốn động từ HTTP cơ bản:

1. **GET:** Lấy dữ liệu (Read).
2. **POST:** Tạo tài nguyên mới (Create).
3. **PUT/PATCH:** Cập nhật tài nguyên hiện có (Update).
4. **DELETE:** Xóa tài nguyên (Delete).

Câu 53: Câu hỏi: Giải thích về mô hình **Client-Server (Khách-Chủ)**. Ưu điểm chính của kiến trúc này là gì?

Lời đáp (Gợi ý): Mô hình **Client-Server** là kiến trúc phân tán trong đó **Client** (ví dụ: trình duyệt web, ứng dụng di động) yêu cầu dịch vụ hoặc tài nguyên từ **Server** (ví dụ: máy chủ cơ sở dữ liệu, máy chủ ứng dụng), và Server phản hồi lại. **Ưu điểm chính:**

1. **Tập trung hóa:** Dữ liệu và tài nguyên được quản lý tập trung trên Server (dễ bảo mật, dễ sao lưu).
2. **Mở rộng:** Dễ dàng mở rộng Server để xử lý nhiều Client hơn.

Câu 54: Câu hỏi: **Socket** là gì? Nó liên quan như thế nào đến lập trình mạng?

Lời đáp (Gợi ý): **Socket** là một điểm cuối (endpoint) của một giao tiếp hai chiều (two-way communication link) giữa hai chương trình đang chạy trên mạng. Trong lập trình mạng,

Socket cung cấp một giao diện lập trình chuẩn cho phép ứng dụng gửi và nhận dữ liệu qua mạng, sử dụng các giao thức như TCP hoặc UDP.

Câu 55: Câu hỏi: Khái niệm **Stateless (Phi trạng thái)** trong REST API là gì? Tại sao tính chất này lại quan trọng?

Lời đáp (Gợi ý): **Stateless** có nghĩa là Server **không lưu trữ** bất kỳ thông tin nào về phiên giao dịch (session) của Client giữa các yêu cầu. Mỗi yêu cầu từ Client phải chứa **tất cả thông tin** cần thiết để Server hiểu và xử lý. **Quan trọng vì:** Giúp dễ dàng thực hiện **cân bằng tải (Load Balancing)** và **mở rộng ngang (Horizontal Scaling)**, vì bất kỳ Server nào cũng có thể xử lý bất kỳ yêu cầu nào từ Client.

Chuyên đề 12: Cơ sở Dữ liệu và DevOps/VCS

Câu 56: Câu hỏi: Sự khác biệt cốt lõi giữa **SQL Database** (ví dụ: PostgreSQL) và **NoSQL Database** (ví dụ: MongoDB) là gì?

Lời đáp (Gợi ý):

- **SQL (Relational):** Dữ liệu được lưu trữ trong các **bảng** có cấu trúc cố định (Schema). Yêu cầu **ACID**. Tốt cho các ứng dụng cần tính nhất quán (Consistency) và giao dịch phức tạp.
- **NoSQL (Non-relational):** Dữ liệu được lưu trữ linh hoạt (ví dụ: Document, Key-Value, Graph). Yêu cầu **BASE** (trong một số trường hợp). Tốt cho dữ liệu lớn (Big Data) và khả năng mở rộng nhanh (Scalability).

Câu 57: Câu hỏi: Giải thích khái niệm **Transaction (Giao dịch)** trong SQL. Tại sao nó lại quan trọng trong các hệ thống tài chính của Viettel?

Lời đáp (Gợi ý): **Transaction** là một tập hợp các thao tác (SQL statements) được coi là một **đơn vị công việc duy nhất và không thể chia cắt**. Nó quan trọng vì nó đảm bảo tính **ACID**, đặc biệt là **Tính nguyên tử (Atomicity)** và **Tính nhất quán (Consistency)**. Ví dụ, trong giao dịch chuyển tiền, tiền phải được trừ ở tài khoản A và cộng vào tài khoản B cùng lúc. Nếu một bước thất bại, toàn bộ giao dịch phải được hoàn tác (rollback).

Câu 58: Câu hỏi: **Docker Image** và **Container** khác nhau như thế nào?

Lời đáp (Gợi ý):

- **Docker Image:** Là một **mẫu (template) tĩnh, chỉ đọc** chứa code, runtime, thư viện, biến môi trường và file cấu hình cần thiết để chạy một ứng dụng. Nó giống như một bản thiết kế.
- **Container:** Là một **thể hiện đang chạy (runtime instance)** của một Image. Nó có thể được chạy, dừng, di chuyển và xóa. Container bao gồm Image và một lớp có thể ghi (writable layer) ở trên cùng.

Câu 59: Câu hỏi: **Git Rebase** khác với **Git Merge** như thế nào? Khi nào bạn nên ưu tiên Rebase?

Lời đáp (Gợi ý):

- **Git Merge:** Kết hợp lịch sử từ nhánh khác bằng cách tạo ra một **commit mới (merge commit)**. Giữ lại lịch sử phát triển chính xác.
- **Git Rebase:** Di chuyển hoặc sắp xếp lại chuỗi các commit của bạn và đặt chúng lên đầu một nhánh khác. Nó **viết lại lịch sử** commit để tạo ra một lịch sử tuyến tính (linear) và sạch sẽ hơn. **Nên ưu tiên Rebase** khi làm việc trên nhánh **cục bộ (local branch)** chưa được chia sẻ (public) để giữ cho lịch sử commit sạch và dễ đọc.

Câu 60: Câu hỏi: Kể tên ba lệnh Git bạn thường sử dụng nhất (ngoài `git add` và `git commit`) và giải thích mục đích của chúng.

Lời đáp (Gợi ý):

1. **git pull:** Tải về và tích hợp những thay đổi mới nhất từ kho lưu trữ từ xa (remote repository) vào nhánh cục bộ hiện tại.
2. **git rebase / git merge:** Để tích hợp các thay đổi từ một nhánh khác vào nhánh hiện tại.
3. **git log:** Xem lịch sử commit của kho lưu trữ.
4. **git stash:** Tạm thời lưu trữ các thay đổi đang thực hiện mà chưa muốn commit, giúp chuyển đổi nhánh dễ dàng.

Chuyên đề 13: Mô hình Thiết kế và Kiến trúc

Câu 61: Câu hỏi: Giải thích mô hình thiết kế **MVC (Model-View-Controller)**. Mỗi thành phần chịu trách nhiệm cho việc gì?

Lời đáp (Gợi ý): MVC là mô hình kiến trúc phần mềm tách biệt các thành phần của ứng dụng thành ba phần có trách nhiệm khác nhau:

1. **Model:** Xử lý logic nghiệp vụ và dữ liệu (data access, validation).
2. **View:** Chịu trách nhiệm về giao diện người dùng (Presentation Logic) và cách dữ liệu được hiển thị.
3. **Controller:** Nhận yêu cầu từ người dùng, gọi Model xử lý, và chọn View để hiển thị kết quả.

Câu 62: Câu hỏi: Khi sử dụng kiến trúc **Microservices**, làm thế nào để các dịch vụ có thể giao tiếp với nhau?

Lời đáp (Gợi ý): Các Microservices giao tiếp chủ yếu qua hai phương thức:

1. **Đồng bộ (Synchronous):** Sử dụng **REST API** (HTTP) hoặc **gRPC** (Remote Procedure Call). Yêu cầu phản hồi ngay lập tức.
2. **Bất đồng bộ (Asynchronous):** Sử dụng **Message Queue** (ví dụ: Kafka, RabbitMQ). Dùng cho các tác vụ không cần phản hồi ngay lập tức hoặc cho phép giao tiếp giữa nhiều dịch vụ.

Câu 63: Câu hỏi: Design Patterns (Mẫu thiết kế) là gì? Nêu một mẫu thiết kế mà anh/chị đã sử dụng (ví dụ: Singleton, Factory) và mục đích của nó.

Lời đáp (Gợi ý): Design Patterns là các giải pháp điển hình, đã được kiểm chứng cho các vấn đề thiết kế phần mềm thường gặp. Chúng cung cấp các khuôn mẫu để giải quyết vấn đề một cách hiệu quả và dễ bảo trì.

- **Ví dụ (Singleton):** Đảm bảo rằng một lớp chỉ có **một thể hiện (instance) duy nhất** trong toàn bộ ứng dụng và cung cấp một điểm truy cập toàn cục cho thể hiện đó. Mục đích: Quản lý các tài nguyên dùng chung (ví dụ: Connection Pool, Logger).

Câu 64: Câu hỏi: Giải thích về Dependency Injection (DI - Tiêm phụ thuộc). Lợi ích chính của DI là gì?

Lời đáp (Gợi ý): DI là một kỹ thuật trong đó một đối tượng (Client) nhận các đối tượng phụ thuộc của nó (Services) từ bên ngoài (thông qua constructor, setter, hoặc field) thay vì tự tạo ra chúng. **Lợi ích:**

1. **Giảm khớp nối (Decoupling):** Các thành phần không bị ràng buộc chặt chẽ với nhau.
2. **Dễ kiểm thử (Testability):** Dễ dàng thay thế các phụ thuộc thực bằng các đối tượng giả (Mocks/Stubs) trong Unit Test.

Câu 65: Câu hỏi: Trình bày về mô hình Event-Driven Architecture (Kiến trúc hướng sự kiện). Nó giải quyết vấn đề gì trong Microservices?

Lời đáp (Gợi ý): Trong Event-Driven Architecture, các thành phần giao tiếp bằng cách phát ra và tiêu thụ **sự kiện (Events)**. Khi một dịch vụ thực hiện một hành động (ví dụ: tạo đơn hàng), nó phát ra một sự kiện, và các dịch vụ khác quan tâm sẽ phản ứng lại. Nó giải quyết vấn đề **khớp nối (coupling)** giữa các Microservices, giúp chúng trở nên độc lập và **linh hoạt hơn** trong việc mở rộng và thay đổi.

Chuyên đề 14: Kỹ năng mềm và Tư duy Viettel

Câu 66: Câu hỏi: Anh/chị sẽ xử lý thế nào nếu một tính năng đã cam kết (committed) không thể hoàn thành đúng hạn (deadline)?

Lời đáp (Gợi ý):

1. **Thông báo sớm:** Ngay lập tức thông báo cho Team Lead/Quản lý dự án về rủi ro và lý do chậm trễ (không giấu diếm).
2. **Đề xuất giải pháp:** Đưa ra các lựa chọn: **Giảm phạm vi (scope)** của tính năng, **Ưu tiên lại** các task khác, hoặc **Yêu cầu hỗ trợ** (thêm tài nguyên).
3. **Lập kế hoạch mới:** Đưa ra một ước tính thời gian hoàn thành thực tế mới và cam kết thực hiện.

Câu 67: Câu hỏi: Viettel là một tập đoàn lớn. Làm thế nào để anh/chị đảm bảo giao tiếp hiệu quả khi làm việc với các phòng ban/team khác có kiến thức chuyên môn khác nhau?

Lời đáp (Gợi ý):

1. **Đơn giản hóa ngôn ngữ:** Tránh sử dụng thuật ngữ kỹ thuật phức tạp (jargon) khi nói chuyện với các bộ phận phi kỹ thuật (kinh doanh, marketing).
2. **Tập trung vào giá trị (Value):** Giải thích công việc theo khía cạnh **giá trị kinh doanh** mà nó mang lại.
3. **Sử dụng trực quan (Visual Aids):** Dùng biểu đồ, sơ đồ kiến trúc để mô tả các khái niệm phức tạp.
4. **Tích cực lắng nghe:** Đảm bảo hiểu rõ nhu cầu của họ trước khi đề xuất giải pháp.

Câu 68: Câu hỏi: Anh/chị đối phó thế nào với những thay đổi đột ngột về yêu cầu (Scope Creep) trong quá trình phát triển dự án?

Lời đáp (Gợi ý):

1. **Ghi nhận và Đánh giá:** Ghi nhận yêu cầu thay đổi và đánh giá cẩn thận **tác động** của nó lên thời gian, chi phí, và nợ kỹ thuật hiện tại.
2. **Đối thoại theo quy trình:** Trao đổi với Product Owner/Quản lý dự án để xác định yêu cầu mới là **cần thiết** hay **mong muốn**.
3. **Thương lượng:** Đề xuất đưa yêu cầu mới vào **Sprint tiếp theo** hoặc **loại bỏ** một tính năng có mức độ ưu tiên thấp hơn khỏi Sprint hiện tại.

Câu 69: Câu hỏi: Theo anh/chị, một **DEV Junior** giỏi cần có những phẩm chất kỹ năng nào ngoài kiến thức code?

Lời đáp (Gợi ý):

1. **Thái độ học hỏi (Growth Mindset):** Sẵn sàng học hỏi công nghệ mới và nhận phản hồi (feedback) tích cực.
2. **Khả năng tự tìm kiếm và giải quyết vấn đề (Self-Sufficiency):** Biết cách tìm kiếm giải pháp trên Google/Stack Overflow và áp dụng.
3. **Tính kỷ luật (Discipline):** Tuân thủ tiêu chuẩn code, quy trình Git và deadline.
4. **Hợp tác (Collaboration):** Khả năng làm việc nhóm và chủ động giao tiếp trong Code Review.

Câu 70: Câu hỏi: Tại sao anh/chị muốn làm việc tại Viettel, và anh/chị thấy văn hóa làm việc của Viettel có gì hấp dẫn?

Lời đáp (Gợi ý): *(Gợi ý trả lời cần liên kết mục tiêu cá nhân với giá trị cốt lõi của Viettel)*
 "Tôi muốn làm việc tại Viettel vì đây là một tập đoàn công nghệ hàng đầu Việt Nam, đi đầu trong các dự án **quốc gia và chuyển đổi số** (digital transformation). Tôi bị hấp dẫn bởi **văn hóa kỷ luật cao và tinh thần tiên phong/khát vọng mãnh liệt (ambition)**. Tôi tin rằng môi trường này sẽ cho phép tôi **học hỏi nhanh chóng** các công nghệ quy mô lớn và đóng góp vào các sản phẩm có **tác động lớn** đến xã hội."

Phần bổ sung (5 Câu hỏi về Tối ưu/Bảo mật Thực tế)

Câu 71: Câu hỏi: JWT (JSON Web Token) được sử dụng như thế nào để xác thực (Authentication) trong kiến trúc Microservices?

Lời đáp (Gợi ý): JWT là một chuỗi token mã hóa chứa thông tin người dùng (Claims). Client gửi username/password để nhận JWT. Sau đó, Client gửi JWT trong header của mỗi yêu cầu API. Gateway hoặc mỗi Microservice có thể **xác minh chữ ký** của token (mà không cần gọi DB) để xác nhận người dùng.

Câu 72: Câu hỏi: Giải thích khái niệm **Idempotency** trong thiết kế API. Tại sao nó quan trọng cho các động từ PUT/DELETE?

Lời đáp (Gợi ý): **Idempotency** nghĩa là việc thực hiện một yêu cầu API **nhiều lần** sẽ cho ra **cùng một kết quả** như khi thực hiện lần đầu. Nó quan trọng cho **PUT/DELETE** vì: Nếu mạng gặp sự cố, Client có thể gửi lại yêu cầu. **PUT** chỉ thay thế tài nguyên (không tạo ra bản sao), và **DELETE** chỉ xóa tài nguyên một lần, ngăn ngừa lỗi trùng lặp dữ liệu hoặc xóa nhầm.

Câu 73: Câu hỏi: Kỹ thuật **Database Sharding (Phân đoạn cơ sở dữ liệu)** là gì? Nó giúp giải quyết vấn đề gì?

Lời đáp (Gợi ý): **Database Sharding** là kỹ thuật **phân chia cơ sở dữ liệu lớn** thành nhiều cơ sở dữ liệu nhỏ hơn, độc lập (gọi là **shards**). Mỗi shard chứa một tập hợp con của toàn bộ dữ liệu. Nó giải quyết vấn đề **khả năng mở rộng (Scalability)** và **hiệu năng**. Khi dữ liệu quá lớn, việc phân mảnh giúp giảm tải I/O cho một máy chủ DB duy nhất.

Câu 74: Câu hỏi: Trong lập trình mạng, khái niệm **Latency (Độ trễ)** và **Throughput (Thông lượng)** khác nhau như thế nào?

Lời đáp (Gợi ý):

- **Latency (Độ trễ):** Là **thời gian** cần thiết để một bit dữ liệu đi từ điểm A đến điểm B (thời gian phản hồi).
- **Throughput (Thông lượng):** Là **số lượng** dữ liệu (hoặc số lượng yêu cầu) có thể được xử lý trong một đơn vị thời gian (ví dụ: megabits/giây, Requests Per Second - RPS).

Câu 75: Câu hỏi: Nêu hai nguyên tắc cơ bản để thiết kế ứng dụng theo triết lý **Twelve-Factor App**.

Lời đáp (Gợi ý):

1. **Codebase (Cơ sở mã):** Một codebase được theo dõi trong VCS (ví dụ: Git) cho mỗi ứng dụng, được dùng để triển khai cho nhiều môi trường (Dev, Prod).
2. **Config (Cấu hình):** Lưu trữ cấu hình **riêng biệt** với code (ví dụ: trong biến môi trường), để dễ dàng thay đổi giữa các môi trường.
3. **Dependencies (Phụ thuộc):** Khai báo và cách ly rõ ràng các thư viện phụ thuộc (ví dụ: dùng `pip freeze` hoặc Maven/Gradle).

Chuyên đề 15: Kiến thức và Văn hóa Viettel

Câu 76: Câu hỏi: Anh/chị hiểu gì về **Sứ mệnh** của Tập đoàn Viettel? Sứ mệnh này liên quan thế nào đến công việc phát triển phần mềm của anh/chị?

Lời đáp (Gợi ý): Sứ mệnh của Viettel là "**Sáng tạo vì con người**" (Creativity for people) hoặc "**Tiên phong kiến tạo xã hội số**". Sứ mệnh này định hướng công việc của DEV: Mọi sản phẩm (ví dụ: ứng dụng My Viettel, hệ thống Viettel Money, giải pháp IoT) phải được thiết kế không chỉ để đạt hiệu quả kỹ thuật mà còn phải **đơn giản, dễ sử dụng** và mang lại **lợi ích thiết thực** cho người dân và xã hội.

Câu 77: Câu hỏi: Văn hóa "**Thách thức và Đột phá**" của Viettel được thể hiện trong công việc phát triển sản phẩm như thế nào?

Lời đáp (Gợi ý): Văn hóa này yêu cầu DEV phải:

1. **Không ngại thử công nghệ mới:** Sẵn sàng áp dụng các kiến trúc hiện đại (AI/ML, 5G, Cloud) để nâng cao chất lượng dịch vụ.
2. **Tư duy làm chủ:** Chịu trách nhiệm hoàn toàn về sản phẩm và không sợ thất bại khi thử nghiệm giải pháp mới.
3. **Khát vọng vươn ra toàn cầu:** Phát triển sản phẩm đạt tiêu chuẩn quốc tế, không chỉ dừng lại ở thị trường Việt Nam.

Câu 78: Câu hỏi: Anh/chị hãy kể tên 2-3 sản phẩm/dịch vụ công nghệ lớn mà Viettel đang phát triển ngoài lĩnh vực viễn thông truyền thống.

Lời đáp (Gợi ý):

1. **Viettel Money:** Hệ sinh thái tài chính số.
2. **Viettel Cloud:** Nền tảng điện toán đám mây cho doanh nghiệp và chính phủ.
3. **Hệ thống AI, IoT:** Các giải pháp giao thông thông minh, y tế điện tử (Telehealth), hoặc các sản phẩm về An toàn thông tin (Cyber Security).

Câu 79: Câu hỏi: Theo anh/chị, Viettel cần phải thay đổi hoặc cải tiến gì để giữ vững vị thế tiên phong trong kỷ nguyên số?

Lời đáp (Gợi ý): Viettel cần tập trung cải tiến vào:

1. **Tăng tốc độ đổi mới:** Giảm thời gian từ khâu nghiên cứu đến triển khai (time-to-market), áp dụng tối đa DevOps/CI/CD.
2. **Trải nghiệm người dùng (UX/UI):** Cải tiến chất lượng sản phẩm theo hướng đơn giản, tiện lợi hơn nữa, học hỏi các công ty công nghệ lớn trên thế giới.
3. **Thu hút và giữ chân nhân tài công nghệ** bằng cơ chế đãi ngộ và môi trường làm việc cởi mở hơn.

Câu 80: Câu hỏi: Anh/chị biết gì về môi trường làm việc và yêu cầu kỷ luật đối với nhân viên công nghệ tại Viettel?

Lời đáp (Gợi ý): Môi trường Viettel được biết đến là **kỷ luật cao** và **áp lực công việc lớn**, nhưng đi kèm với cơ hội tham gia các **dự án quy mô quốc gia** và quốc tế. Yêu cầu kỷ luật thể hiện qua: **Tác phong chuyên nghiệp, tuân thủ quy trình nghiêm ngặt** (đặc biệt trong

các dự án an ninh, quốc phòng), và **tinh thần làm việc cường độ cao** để đáp ứng các mục tiêu đột phá.

Chuyên đề 16: Định hướng Công việc và Mục tiêu Cá nhân

Câu 81: Câu hỏi: Anh/chị thấy mình ở vị trí nào trong 3 năm nữa nếu làm việc tại Viettel?

Lời đáp (Gợi ý): "Trong 3 năm tới, tôi muốn trở thành một **Senior/Expert Developer** trong lĩnh vực [Nêu rõ lĩnh vực: Back-end/Data/Cloud]. Tôi không chỉ muốn viết code, mà còn muốn tham gia vào **quá trình thiết kế kiến trúc hệ thống (architecture design)** và **hướng dẫn (mentor)** các thành viên mới của team, góp phần vào thành công chung của đơn vị."

Câu 82: Câu hỏi: Anh/chị quan tâm nhất đến khía cạnh nào trong công việc: **Tiền lương, Cơ hội học hỏi**, hay **Vị trí/Danh vọng**?

Lời đáp (Gợi ý): *(Cần thể hiện sự cân bằng)* "Cả ba đều quan trọng, nhưng tôi ưu tiên **Cơ hội học hỏi và phát triển** (thứ nhất) và **Tiền lương xứng đáng** (thứ hai). Tại Viettel, tôi kỳ vọng được làm việc với các công nghệ quy mô lớn và thử thách bản thân để thăng tiến lên các vị trí lãnh đạo kỹ thuật, từ đó, cả thu nhập và danh vọng sẽ là kết quả tự nhiên."

Câu 83: Câu hỏi: Anh/chị đã chuẩn bị gì để làm quen với codebase (mã nguồn) và quy trình làm việc phức tạp của Viettel?

Lời đáp (Gợi ý):

1. **Học hỏi chủ động:** Dành thời gian ngoài giờ để đọc tài liệu, tìm hiểu quy trình CI/CD và kiến trúc của hệ thống hiện tại.
2. **Tham gia Pair Programming:** Tích cực đề nghị làm việc cặp với các thành viên kỳ cựu để hiểu rõ logic nghiệp vụ.
3. **Thực hiện các task nhỏ:** Bắt đầu bằng các task đơn giản (bug fixes, refactoring nhỏ) để làm quen với codebase trước khi xử lý các tính năng lớn.

Câu 84: Câu hỏi: Anh/chị nghĩ mình có thể đóng góp gì khác biệt cho đội ngũ phát triển tại Viettel so với các ứng viên khác?

Lời đáp (Gợi ý): "Ngoài kiến thức chuyên môn, tôi có kinh nghiệm đặc biệt về [Nêu 1 kỹ năng độc đáo: DevSecOps/Tối ưu hiệu năng Database/Front-end hiện đại]. Tôi tin rằng tôi có thể mang lại một **góc nhìn mới** về [Nêu lĩnh vực đó] và giúp team áp dụng các thực hành tốt nhất để cải thiện chất lượng sản phẩm và giảm **Technical Debt**."

Câu 85: Câu hỏi: Anh/chị xử lý thế nào khi phải làm việc trên một dự án mà công nghệ (stack) đã cũ và khó bảo trì?

Lời đáp (Gợi ý):

1. **Phân tích rủi ro:** Đánh giá mức độ rủi ro và chi phí bảo trì của code cũ.

2. **Đề xuất chiến lược Refactoring (Tái cấu trúc):** Đề xuất với team/quản lý về việc dành một phần trăm thời gian của Sprint (ví dụ: 10-20%) để từng bước di chuyển (migration) sang công nghệ mới, bắt đầu từ các module quan trọng nhất.
3. **Ưu tiên ổn định:** Trong thời gian đó, đảm bảo tính năng mới được thêm vào vẫn hoạt động ổn định và tuân thủ các quy tắc code hiện tại.

Chuyên đề 17: Thương lượng Lương và Đãi ngộ (Salary Negotiation)

Câu 86: Câu hỏi: Mức lương mong muốn của anh/chị cho vị trí này là bao nhiêu (Gross Income)?

Lời đáp (Gợi ý): (Trả lời linh hoạt, đưa ra một **phạm vi** thay vì con số cố định, và khẳng định giá trị bản thân.) "Dựa trên kinh nghiệm [**X năm**] và mức lương thị trường cho vị trí tương đương tại các tập đoàn lớn như Viettel, tôi kỳ vọng một mức lương trong khoảng [**A triệu VNĐ**] đến [**B triệu VNĐ**] Gross. Tôi tin rằng kỹ năng [**Kỹ năng chính**] của tôi sẽ mang lại giá trị tương xứng cho dự án."

Câu 87: Câu hỏi: Con số này có thể thương lượng được không? Nếu chúng tôi chỉ có thể đáp ứng X% (thấp hơn kỳ vọng) mức lương của anh/chị, anh/chị sẽ chấp nhận không?

Lời đáp (Gợi ý): "Mức lương đưa ra là mức tôi kỳ vọng dựa trên năng lực. Tuy nhiên, tôi rất quan tâm đến cơ hội tại Viettel. Nếu mức lương thấp hơn, tôi muốn được cân nhắc thêm về các yếu tố đãi ngộ khác như: **Cơ hội thăng tiến rõ ràng**, **Tỷ lệ thưởng dự án/năm**, hoặc **Các phúc lợi về bảo hiểm/đào tạo chuyên sâu**."

Câu 88: Câu hỏi: Ngoài lương, anh/chị mong muốn những phúc lợi nào khác từ Viettel?

Lời đáp (Gợi ý):

1. **Cơ hội đào tạo chuyên sâu (Training):** Được tham gia các khóa học về Cloud (AWS/Azure) hoặc AI/Big Data.
2. **Phụ cấp:** Phụ cấp điện thoại, đi lại, hoặc ăn trưa hợp lý.
3. **Cân bằng công việc-cuộc sống:** Chính sách làm việc linh hoạt (flexible hours) nếu tính chất công việc cho phép.
4. **Chính sách thưởng:** Thưởng dự án/thưởng cuối năm rõ ràng, minh bạch.

Câu 89: Câu hỏi: Tại sao mức lương hiện tại của anh/chị lại là [X]? Anh/chị nghĩ mức tăng trưởng nào là hợp lý khi chuyển sang Viettel?

Lời đáp (Gợi ý): "Mức lương hiện tại phản ánh kinh nghiệm của tôi tại môi trường [Nêu đặc điểm môi trường cũ: startup/product company]. Tôi tìm kiếm mức tăng trưởng hợp lý không chỉ vì chuyển công ty, mà vì **quy mô dự án và thách thức kỹ thuật** tại Viettel lớn hơn đáng kể. Mức tăng trưởng khoảng [**Nêu tỷ lệ, ví dụ: 20-30%**] sẽ phù hợp với sự phát triển sự nghiệp của tôi tại một tập đoàn hàng đầu."

Câu 90: Câu hỏi: Viettel yêu cầu cam kết làm việc trong 2 năm. Anh/chị có sẵn lòng cam kết không?

Lời đáp (Gợi ý): "Hoàn toàn sẵn lòng. Tôi coi việc cam kết 2 năm là một cơ hội để tôi thực sự **đóng góp vào các dự án lớn và phát triển chuyên môn sâu sắc**. Nếu cả hai bên đạt được thỏa thuận về mục tiêu công việc và đãi ngộ, tôi cam kết gắn bó và nỗ lực hết mình trong thời gian này."

Chuyên đề 18: Xử lý Tình huống Bẫy (Trap Questions)

Câu 91: Câu hỏi (Trap): Anh/chị đã từng bắt đồng nghiệp trọng với đồng nghiệp hoặc quản lý về mặt kỹ thuật chưa? Anh/chị đã giải quyết nó như thế nào?

Lời đáp (Gợi ý): *(Không đổ lỗi, tập trung vào quy trình và mục tiêu chung)* "Có, bắt đồng là điều không thể tránh khỏi. Tôi đã từng bắt đồng về việc lựa chọn giữa **[Công nghệ A]** và **[Công nghệ B]** cho tính năng **[Tên tính năng]**. Tôi đã giải quyết bằng cách: **Trình bày dữ liệu** (phân tích hiệu năng, chi phí), **tìm một bên thứ ba trung lập** (Tech Lead/Architect) để tham khảo, và cuối cùng, **chấp nhận quyết định của team** dựa trên mục tiêu chung là tốc độ triển khai/khả năng bảo trì."

Câu 92: Câu hỏi (Trap): Anh/chị sẽ làm gì nếu phát hiện ra một **lỗi nghiêm trọng (Critical Bug)** trong code của đồng nghiệp đã được deploy (triển khai) lên Production?

Lời đáp (Gợi ý):

1. **Hành động khẩn cấp:** Ngay lập tức thông báo cho team lead/sếp, và nếu có thể, thực hiện **Rollback** (khôi phục) phiên bản code trước đó để dừng thiệt hại.
2. **Phân tích và sửa chữa:** Cùng đồng nghiệp (hoặc một người khác) phân tích nhanh nguyên nhân gốc (root cause) và tạo bản vá lỗi gấp.
3. **Học hỏi:** Sau khi vá lỗi, đề xuất cải tiến quy trình **Code Review** hoặc thêm **Integration Test** để ngăn lỗi tương tự xảy ra.

Câu 93: Câu hỏi (Trap): Anh/chị sẽ làm thêm giờ (OT) bao lâu một tuần nếu dự án yêu cầu, và anh/chị nghĩ gì về việc này?

Lời đáp (Gợi ý): *(Thể hiện sự cam kết nhưng có giới hạn)* "Tôi hiểu rằng các dự án quan trọng tại Viettel đôi khi đòi hỏi sự nỗ lực cao độ. Tôi sẵn sàng làm việc ngoài giờ khi cần thiết, đặc biệt là trong các giai đoạn **triển khai (deployment)** hoặc **khắc phục sự cố nghiêm trọng**. Tuy nhiên, tôi tin rằng làm việc **hiệu quả** trong giờ hành chính quan trọng hơn làm việc **quá nhiều**. Nếu việc OT kéo dài thường xuyên, tôi sẽ đề xuất với quản lý xem xét lại **phạm vi công việc** hoặc **phân bổ tài nguyên** để đảm bảo chất lượng và sức khỏe lâu dài."

Câu 94: Câu hỏi (Trap): Anh/chị đã từng bỏ dở (abandon) một dự án nào chưa? Lý do là gì?

Lời đáp (Gợi ý): *(Chọn câu chuyện có trách nhiệm, không phải vì khó khăn cá nhân)* "Tôi chưa từng bỏ dở dự án mà không bàn giao. Tuy nhiên, tôi đã từng đề xuất **dừng phát triển** một tính năng. Lý do là sau khi nghiên cứu thị trường, chúng tôi nhận ra tính năng đó **không mang lại giá trị kinh doanh** hoặc **công nghệ chưa khả thi** trong khung thời gian cho phép. Tôi luôn ủng hộ việc đưa ra quyết định dựa trên **dữ liệu thực tế** để đảm bảo nguồn lực được sử dụng hiệu quả nhất."

Câu 95: Câu hỏi (Trap): Anh/chị có câu hỏi nào dành cho chúng tôi không?

Lời đáp (Gợi ý): *(Luôn hỏi để thể hiện sự quan tâm nghiêm túc)*

1. "Quy trình **Code Review** và **triển khai (deployment)** cho vị trí này tại Viettel được thực hiện như thế nào?"
2. "Đơn vị [Tên đơn vị ứng tuyển] đang sử dụng **Framework/Công nghệ Cloud** nào là chủ yếu trong các dự án mới?"
3. "Viễn cảnh 6-12 tháng tới, **thách thức kỹ thuật lớn nhất** mà team đang phải đối mặt là gì?"